

Table of Contents

1. [Introducing Node.js](#)
2. [Understanding Node Usefulness](#)
3. [Working with Inbuilt Node Modules](#)
4. [Sharing Code Between JS Files](#)
5. [Understanding Event Loop](#)
6. [Creating a Simple Server](#)

Introducing Node.js

Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine. It allows developers to run JavaScript on the server side, outside of a web browser.

Key Characteristics

Node.js was created by Ryan Dahl in 2009 and has since become one of the most popular platforms for building server-side applications. Unlike traditional JavaScript that runs in browsers, Node.js enables JavaScript to interact with the operating system, file system, and network.

Core Features:

- Runs JavaScript on the server
- Built on V8 engine (same engine that powers Google Chrome)
- Event-driven, non-blocking I/O model
- Single-threaded with event loop architecture
- Cross-platform (Windows, Linux, macOS)

Installing Node.js

Visit the official website at nodejs.org and download the LTS (Long Term Support) version for your operating system. After installation, verify by opening your terminal or command prompt and typing:

```
bash
node --version
npm --version
```

Your First Node Program

Create a file named hello.js with the following content:

```
javascript
```

```
console.log("Hello from Node.js!");  
console.log("Node version:", process.version);
```

Run it using:

```
bash
```

```
node hello.js
```

Understanding Node Usefulness

Node.js has become essential in modern web development due to its unique advantages and capabilities.

Why Use Node.js?

JavaScript Everywhere: Developers can use the same language (JavaScript) for both frontend and backend development, reducing context switching and enabling code reuse between client and server.

High Performance: The V8 engine compiles JavaScript to native machine code, making execution extremely fast. The non-blocking I/O model allows Node.js to handle thousands of concurrent connections efficiently.

Rich Ecosystem: NPM (Node Package Manager) provides access to over a million packages, making it easy to add functionality to your applications without reinventing the wheel.

Scalability: Node.js is designed to build scalable network applications. Its event-driven architecture makes it ideal for real-time applications that require handling multiple concurrent requests.

Common Use Cases

Node.js excels in building real-time applications such as chat applications, collaborative tools, and live streaming platforms. It is widely used for creating RESTful APIs and microservices architectures. Many companies use Node.js for building command-line tools and utilities. It powers server-side rendering for modern frontend frameworks like React and Vue.js. Data streaming applications and IoT (Internet of Things) projects also benefit from Node.js's efficient handling of data flows.

Industry Adoption

Major companies including Netflix, LinkedIn, Uber, PayPal, NASA, and Walmart use Node.js in their production systems, demonstrating its reliability and scalability for enterprise applications.

Working with Inbuilt Node Modules

Node.js comes with several built-in modules that provide essential functionality without requiring

external packages. These modules are part of the Node.js core and are always available.

The HTTP Module

The `http` module enables Node.js to create HTTP servers and make HTTP requests.

javascript

```
const http = require('http');

// Creating a basic server
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World!');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Making HTTP Requests:

javascript

```
const http = require('http');

const options = {
  hostname: 'jsonplaceholder.typicode.com',
  path: '/posts/1',
  method: 'GET'
};

const req = http.request(options, (res) => {
  let data = '';

  res.on('data', (chunk) => {
    data += chunk;
  });

  res.on('end', () => {
    console.log(JSON.parse(data));
  });
});

req.on('error', (error) => {
  console.error(error);
});

req.end();
```

The FS (File System) Module

The fs module provides methods for interacting with the file system, allowing you to read, write, delete, and manipulate files and directories.

Reading Files:

javascript

```
const fs = require('fs');

// Synchronous reading
const dataSync = fs.readFileSync('example.txt', 'utf8');
console.log(dataSync);

// Asynchronous reading (preferred)
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  console.log(data);
});
```

Writing Files:

javascript

```
const fs = require('fs');

// Writing to a file
const content = 'This is sample content for our file.';

fs.writeFile('output.txt', content, (err) => {
  if (err) {
    console.error('Error writing file:', err);
    return;
  }
  console.log('File written successfully!');
});

// Appending to a file
fs.appendFile('output.txt', '\nThis is appended text.', (err) => {
  if (err) {
    console.error('Error appending to file:', err);
    return;
  }
  console.log('Content appended!');
});
```

Working with Directories:

javascript

```

const fs = require('fs');

// Creating a directory
fs.mkdir('new-folder', (err) => {
  if (err) {
    console.error('Error creating directory:', err);
    return;
  }
  console.log('Directory created!');
});

// Reading directory contents
fs.readdir('.', (err, files) => {
  if (err) {
    console.error('Error reading directory:', err);
    return;
  }
  console.log('Files in current directory:', files);
});

// Deleting a file
fs.unlink('unwanted-file.txt', (err) => {
  if (err) {
    console.error('Error deleting file:', err);
    return;
  }
  console.log('File deleted!');
});

```

The Path Module

The path module provides utilities for working with file and directory paths in a cross-platform way.

javascript

```

const path = require('path');

// Joining path segments
const fullPath = path.join('/users', 'john', 'documents', 'file.txt');
console.log(fullPath); // /users/john/documents/file.txt

// Getting file extension
const ext = path.extname('document.pdf');
console.log(ext); // .pdf

// Getting filename without extension
const basename = path.basename('folder/file.txt', '.txt');
console.log(basename); // file

```

```

// Getting directory name
const dirname = path.dirname('/users/john/file.txt');
console.log(dirname); // /users/john

// Resolving absolute path
const absolutePath = path.resolve('folder', 'file.txt');
console.log(absolutePath); // Absolute path based on current working
directory

// Parsing a path
const parsedPath = path.parse('/home/user/dir/file.txt');
console.log(parsedPath);
/* Output:
{
  root: '/',
  dir: '/home/user/dir',
  base: 'file.txt',
  ext: '.txt',
  name: 'file'
}
*/

```

The OS Module

The os module provides operating system-related utility methods and properties.

javascript

```

const os = require('os');

// Getting system information
console.log('Platform:', os.platform()); // linux, darwin, win32, etc.
console.log('Architecture:', os.arch()); // x64, arm, etc.
console.log('CPU Cores:', os.cpus().length);
console.log('Total Memory:', os.totalmem() / (1024 ** 3), 'GB');
console.log('Free Memory:', os.freemem() / (1024 ** 3), 'GB');
console.log('Home Directory:', os.homedir());
console.log('Temp Directory:', os.tmpdir());
console.log('Hostname:', os.hostname());
console.log('Uptime:', os.uptime(), 'seconds');

// Getting network interfaces
console.log('Network Interfaces:', os.networkInterfaces());

// Getting user info
console.log('User Info:', os.userInfo());

```

Practical Example - System Monitor:

javascript

```
const os = require('os');

function getSystemStats() {
  const totalMem = os.totalmem() / (1024 ** 3);
  const freeMem = os.freemem() / (1024 ** 3);
  const usedMem = totalMem - freeMem;
  const memUsagePercent = (usedMem / totalMem) * 100;

  return {
    platform: os.platform(),
    cpuCount: os.cpus().length,
    totalMemory: totalMem.toFixed(2) + ' GB',
    freeMemory: freeMem.toFixed(2) + ' GB',
    usedMemory: usedMem.toFixed(2) + ' GB',
    memoryUsage: memUsagePercent.toFixed(2) + '%',
    uptime: (os.uptime() / 3600).toFixed(2) + ' hours'
  };
}

console.log('System Statistics:');
console.log(getSystemStats());
```

Sharing Code Between JS Files

As applications grow, organizing code into multiple files becomes essential for maintainability. Node.js provides the CommonJS module system for sharing code between files.

Module Basics

Every file in Node.js is treated as a separate module. Each module has its own scope, meaning variables and functions defined in one file are not automatically available in another.

Exporting from Modules

There are several ways to export code from a module.

Method 1: Using `module.exports`

Create a file named `calculator.js`:

javascript

```
// calculator.js
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
```

```

        return a - b;
    }

    function multiply(a, b) {
        return a * b;
    }

    function divide(a, b) {
        if (b === 0) {
            throw new Error('Cannot divide by zero');
        }
        return a / b;
    }

    // Exporting multiple functions
    module.exports = {
        add,
        subtract,
        multiply,
        divide
    };

```

Method 2: Exporting Individual Items

```

javascript
// utils.js
exports.greeting = function(name) {
    return `Hello, ${name}!`;
};

exports.farewell = function(name) {
    return `Goodbye, ${name}!`;
};

exports.PI = 3.14159;

```

Method 3: Exporting a Single Function or Class

```

javascript
// logger.js
module.exports = function log(message) {
    const timestamp = new Date().toISOString();
    console.log(`[${timestamp}] ${message}`);
};

```

Importing Modules

Use the `require()` function to import modules.

```

javascript

```



```
// main.js
const calculator = require('./calculator');
const utils = require('./utils');
const log = require('./logger');

// Using imported modules
console.log('Addition:', calculator.add(5, 3));
console.log('Subtraction:', calculator.subtract(10, 4));

console.log(utils.greeting('Student'));
console.log('PI value:', utils.PI);

log('Application started');
```

Building a Practical Example

Let's create a simple user management system with multiple modules.

File: user.js

javascript

```
// user.js
class User {
  constructor(name, email, age) {
    this.name = name;
    this.email = email;
    this.age = age;
    this.createdAt = new Date();
  }

  getInfo() {
    return `${this.name} (${this.email}), Age: ${this.age}`;
  }

  isAdult() {
    return this.age >= 18;
  }
}

module.exports = User;
```

File: userManager.js

javascript

```
// userManager.js
const User = require('./user');

class UserManager {
  constructor() {
    this.users = [];
  }
}
```

```

    }

    addUser(name, email, age) {
        const user = new User(name, email, age);
        this.users.push(user);
        return user;
    }

    getUsers() {
        return this.users;
    }

    findUserByEmail(email) {
        return this.users.find(user => user.email === email);
    }

    getAdultUsers() {
        return this.users.filter(user => user.isAdult());
    }

    getUserCount() {
        return this.users.length;
    }
}

module.exports = UserManager;

```

File: app.js

javascript

// app.js

```
const UserManager = require('./userManager');
```

```
const manager = new UserManager();
```

// Adding users

```
manager.addUser('Alice Johnson', 'alice@example.com', 25);
```

```
manager.addUser('Bob Smith', 'bob@example.com', 17);
```

```
manager.addUser('Charlie Brown', 'charlie@example.com', 30);
```

// Getting all users

```
console.log('All Users:');
```

```
manager.getUsers().forEach(user => {
```

```
    console.log(user.getInfo());
```

```
});
```

// Getting adult users

```
console.log('\nAdult Users:');
```

```
manager.getAdultUsers().forEach(user => {
```

```

        console.log(user.getInfo());
    });

    // Finding a specific user
    const foundUser = manager.findUserByEmail('alice@example.com');
    console.log('\nFound User:', foundUser ? foundUser.getInfo() : 'Not
found');

    console.log('\nTotal Users:', manager.getUserCount());

```

Module Caching

Node.js caches modules after the first time they are loaded. This means if you require the same module multiple times, Node.js will return the cached version rather than re-executing the module code.

javascript

```

// counter.js
let count = 0;

module.exports = {
  increment() {
    count++;
  },
  getCount() {
    return count;
  }
};

```

javascript

```

// app.js
const counter1 = require('./counter');
const counter2 = require('./counter');

counter1.increment();
counter1.increment();

console.log(counter1.getCount()); // 2
console.log(counter2.getCount()); // 2 (same instance due to caching)

```

Understanding Event Loop

The event loop is the heart of Node.js's asynchronous, non-blocking architecture. Understanding it is crucial for writing efficient Node.js applications.

What is the Event Loop?

The event loop is a mechanism that allows Node.js to perform non-blocking I/O operations despite JavaScript being single-threaded. It continuously checks for and executes tasks from various queues.

How JavaScript Execution Works

JavaScript is single-threaded, meaning it can only execute one piece of code at a time. However, Node.js can handle thousands of concurrent operations through asynchronous callbacks and the event loop.

The Call Stack: The call stack is where JavaScript keeps track of function calls. When a function is called, it's added to the stack. When it returns, it's removed from the stack.

```
javascript
function first() {
    console.log('First function');
    second();
    console.log('First function end');
}

function second() {
    console.log('Second function');
}

first();

// Output:
// First function
// Second function
// First function end
```

Event Loop Phases

The event loop operates in phases, each handling specific types of callbacks.

1. **Timers Phase:** Executes callbacks scheduled by `setTimeout()` and `setInterval()`
2. **Pending Callbacks Phase:** Executes I/O callbacks deferred from the previous cycle
3. **Idle, Prepare Phase:** Internal use only
4. **Poll Phase:** Retrieves new I/O events and executes I/O callbacks
5. **Check Phase:** Executes `setImmediate()` callbacks
6. **Close Callbacks Phase:** Executes close event callbacks

Microtasks and Macrotasks

Microtasks include Promise callbacks and `process.nextTick()`. These are executed before moving to the next phase of the event loop.

Macrotasks include `setTimeout`, `setInterval`, `setImmediate`, and I/O operations. These are

executed according to the event loop phases.

javascript

```
console.log('Start');

setTimeout(() => {
  console.log('Timeout');
}, 0);

Promise.resolve().then(() => {
  console.log('Promise');
});

process.nextTick(() => {
  console.log('Next Tick');
});

console.log('End');

// Output:
// Start
// End
// Next Tick
// Promise
// Timeout
```

Explanation: Synchronous code (Start, End) executes first. Then microtasks execute in order: process.nextTick() runs before Promise callbacks. Finally, the setTimeout callback (macrotask) executes.

Blocking vs Non-Blocking Operations

Blocking Code Example:

javascript

```
const fs = require('fs');

console.log('Start');

// Blocking - synchronous
const data = fs.readFileSync('large-file.txt', 'utf8');
console.log('File read complete');

console.log('End');

// The entire program waits for file reading to complete
```

Non-Blocking Code Example:

javascript

```
const fs = require('fs');
```

```

console.log('Start');

// Non-blocking - asynchronous
fs.readFile('large-file.txt', 'utf8', (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('File read complete');
});

console.log('End');

// Output:
// Start
// End
// File read complete (when ready)

```

Practical Example: Understanding Execution Order

```

javascript
const fs = require('fs');

console.log('1: Start');

setTimeout(() => {
  console.log('2: setTimeout 0ms');
}, 0);

setImmediate(() => {
  console.log('3: setImmediate');
});

process.nextTick(() => {
  console.log('4: nextTick');
});

Promise.resolve().then(() => {
  console.log('5: Promise');
});

fs.readFile(__filename, () => {
  console.log('6: File I/O');

  setTimeout(() => {
    console.log('7: setTimeout inside I/O');
  });
});

```

```

    }, 0);

    setImmediate(() => {
        console.log('8: setImmediate inside I/O');
    });

    process.nextTick(() => {
        console.log('9: nextTick inside I/O');
    });
});

```

```

console.log('10: End');

```

```

// Typical Output:
// 1: Start
// 10: End
// 4: nextTick
// 5: Promise
// 2: setTimeout 0ms
// 3: setImmediate
// 6: File I/O
// 9: nextTick inside I/O
// 8: setImmediate inside I/O
// 7: setTimeout inside I/O

```

Best Practices for Event Loop

Avoid blocking the event loop with CPU-intensive synchronous operations. Break large tasks into smaller chunks using `setImmediate()` or `process.nextTick()`. Use asynchronous versions of methods whenever possible (`readFile` instead of `readFileSync`). Be careful with `process.nextTick()` as it can starve the event loop if used recursively without conditions. Use worker threads for CPU-intensive tasks to prevent blocking the main thread.

Creating a Simple Server

Creating HTTP servers is one of the most common uses of Node.js. Let's build servers with increasing complexity.

Basic HTTP Server

```

javascript
const http = require('http');

const server = http.createServer((req, res) => {
    res.statusCode = 200;

```

```

    res.setHeader('Content-Type', 'text/plain');
    res.end('Hello, World!\n');
  });

const PORT = 3000;

server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});

```

Run this code and visit <http://localhost:3000> in your browser to see the result.

Understanding Request and Response Objects

The request object (req) contains information about the incoming HTTP request, while the response object (res) is used to send data back to the client.

javascript

```

const http = require('http');

const server = http.createServer((req, res) => {
  console.log('Request Method:', req.method);
  console.log('Request URL:', req.url);
  console.log('Request Headers:', req.headers);

  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('<h1>Request Information</h1>');
  res.write(`<p>Method: ${req.method}</p>`);
  res.write(`<p>URL: ${req.url}</p>`);
  res.end();
});

server.listen(3000, () => {
  console.log('Server listening on port 3000');
});

```

Routing Requests

Handle different URLs with routing logic.

javascript

```

const http = require('http');

const server = http.createServer((req, res) => {
  const url = req.url;

  if (url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Home Page</h1><p>Welcome to the home page!</p>');
  }
});

```



```

    } else if (url === '/about') {
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end('<h1>About Page</h1><p>This is the about page.</p>');
    } else if (url === '/contact') {
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end('<h1>Contact Page</h1><p>Contact us at
contact@example.com</p>');
    } else {
      res.writeHead(404, { 'Content-Type': 'text/html' });
      res.end('<h1>404 - Page Not Found</h1>');
    }
  });

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});

```

Serving JSON Data

Modern web applications often work with JSON data.

javascript

```

const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/api/users' && req.method === 'GET') {
    const users = [
      { id: 1, name: 'Alice', email: 'alice@example.com' },
      { id: 2, name: 'Bob', email: 'bob@example.com' },
      { id: 3, name: 'Charlie', email: 'charlie@example.com' }
    ];

    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(users));
  } else if (req.url === '/api/status' && req.method === 'GET') {
    const status = {
      status: 'OK',
      timestamp: new Date().toISOString(),
      uptime: process.uptime()
    };

    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(status));
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Not Found' }));
  }
});

```

```
server.listen(3000, () => {
  console.log('API server running on http://localhost:3000');
});
```

Serving Static Files

Serve HTML, CSS, and JavaScript files from the file system.

javascript

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
  let filePath = '.' + req.url;
  if (filePath === './') {
    filePath = './index.html';
  }

  const extname = path.extname(filePath);

  const contentType = {
    '.html': 'text/html',
    '.css': 'text/css',
    '.js': 'text/javascript',
    '.json': 'application/json',
    '.png': 'image/png',
    '.jpg': 'image/jpeg',
    '.gif': 'image/gif'
  };

  const contentType = contentType[extname] || 'text/plain';

  fs.readFile(filePath, (err, content) => {
    if (err) {
      if (err.code === 'ENOENT') {
        res.writeHead(404, { 'Content-Type': 'text/html' });
        res.end('<h1>404 - File Not Found</h1>');
      } else {
        res.writeHead(500);
        res.end('Server Error: ' + err.code);
      }
    } else {
      res.writeHead(200, { 'Content-Type': contentType });
      res.end(content);
    }
  });
});
```

```
});

server.listen(3000, () => {
  console.log('Static file server running on
http://localhost:3000');
});
```

Complete Example: RESTful API Server

Here's a complete example combining multiple concepts.

```
javascript
const http = require('http');
const url = require('url');

// In-memory data store
let students = [
  { id: 1, name: 'Alice Johnson', grade: 'A', age: 20 },
  { id: 2, name: 'Bob Smith', grade: 'B', age: 21 }
];

let nextId = 3;

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true);
  const pathname = parsedUrl.pathname;
  const method = req.method;

  // Set CORS headers
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT,
DELETE');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  // GET all students
  if (pathname === '/api/students' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(students));
  }
  // GET single student
  else if (pathname.match(/\/api\/students\/\d+\/) && method ===
'GET') {
    const id = parseInt(pathname.split('/')[3]);
    const student = students.find(s => s.id === id);

    if (student) {
      res.writeHead(200, { 'Content-Type': 'application/json'
});
    }
  }
});
```

```

        res.end(JSON.stringify(student));
    } else {
        res.writeHead(404, { 'Content-Type': 'application/json'
    });
        res.end(JSON.stringify({ error: 'Student not found' }));
    }
}
// POST new student
else if (pathname === '/api/students' && method === 'POST') {
    let body = '';

    req.on('data', chunk => {
        body += chunk.toString();
    });

    req.on('end', () => {
        const newStudent = JSON.parse(body);
        newStudent.id = nextId++;
        students.push(newStudent);

        res.writeHead(201, { 'Content-Type': 'application/json'
    });
        res.end(JSON.stringify(newStudent));
    });
}
// DELETE student
else if (pathname.match(/\/api\/students\/\d+/) && method ===
'DELETE') {
    const id = parseInt(pathname.split('/')[3]);
    const index = students.findIndex(s => s.id === id);

    if (index !== -1) {
        students.splice(index, 1);
        res.writeHead(200, { 'Content-Type': 'application/json'
    });
        res.end(JSON.stringify({ message: 'Student deleted' }));
    } else {
        res.writeHead(404, { 'Content-Type': 'application/json'
    });
        res.end(JSON.stringify({ error: 'Student not found' }));
    }
}
// 404 for unknown routes
else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Route not found' }));
}
});

```

```
const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Student API server running on
http://localhost:${PORT}`);
  console.log('Available endpoints:');
  console.log('  GET    /api/students');
  console.log('  GET    /api/students/:id');
  console.log('  POST   /api/students');
  console.log('  DELETE /api/students/:id');
});
```

Testing Your Server

You can test the API using curl commands in your terminal:

bash

Get all students

```
curl http://localhost:3000/api/students
```

Get single student

```
curl http://localhost:3000/api/students/1
```

Add new student

```
curl -X POST http://localhost:3000/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Charlie Brown","grade":"A","age":22}'
```

Delete student

```
curl -X DELETE http://localhost:3000/api/students/2
```

Practice Exercises

Exercise 1: File Operations

Create a Node.js program that reads a text file, counts the number of words, and writes the count to a new file.

Exercise 2: Custom Module

Build a module called stringUtils.js that exports functions for capitalizing strings, reversing strings, and counting vowels.

Exercise 3: System Information Logger

Create an application that logs system information (CPU, memory, platform) to a file every 5 seconds using the `os` and `fs` modules.

Exercise 4: Simple API

Build a simple TODO API with endpoints for creating, reading, updating, and deleting tasks. Store tasks in memory.

Exercise 5: Event Loop

Write a program demonstrating the execution order of `setTimeout`, `setImmediate`, `process.nextTick`, and Promise callbacks.